

Clasificación no supervisada: K-Means y SOM

Haessler Joan Ortiz Moncada

Universidad Distrital Francisco José de Caldas

Facultad de Ingeniería

Curso: Big Data

23 de septiembre de 2025

1. Introducción

Este documento presenta el desarrollo de los ejercicios planteados en el **Taller_I** del curso de Big Data (aunque, cronológicamente, corresponde al número 5 del curso), documentando los hallazgos y análisis correspondientes. Este taller se centra principalmente en el abordaje de dos *datasets*: uno proporcionado por el profesor de la clase, denominado `data_clusters.mat`, y otro descargado de [Avila's dataset](#). A partir de estos conjuntos de datos se busca implementar y practicar metodologías de clasificación no supervisada, específicamente K-Means y los mapas autoorganizados (SOM, por sus siglas en inglés).

2. Metodología

En cuanto al orden, el trabajo se desarrolla conforme a lo establecido en el **Taller_I**. Respecto a la forma, la implementación de los modelos, la preparación de los datos y la visualización de los resultados se llevan a cabo mediante un archivo de **Jupyter Notebook** desarrollado en Colab. Cabe aclarar que la Inteligencia Artificial (IA) Gemini asistió en la generación del código requerido. Por último, el código producido, los datos implementados y el informe generado, pueden consultarse en la carpeta `taller_5` del repositorio [bigdata_repositorio](#) de GitHub.

3. Desarrollo

El **Taller_I** consta de dos ítems a desarrollar. Como es lógico, esta sección abordará en primer lugar el primer ítem y, posteriormente, el segundo. Para el primer ítem se requiere implementar los modelos K-Means y SOM sobre el dataset `data_clusters.mat`, se permite una experimentación libre orientada únicamente a identificar los *clusters* y la retícula de las neuronas, respectivamente.

3.1. KMeans con `data_clusters.mat`

Teniendo el dataset `data_clusters.mat`, se procede a etiquetar sus dos columnas o características como `columnaA` y `columnaB`, así como a medir su tamaño, el cual corres-

ponde a 135×2 , donde ambas variables son de tipo `float`. Como regla general en este tipo de ejercicios, es recomendable estandarizar los datos para evitar problemas de escala entre las variables; sin embargo, debido a que los dominios de las características son relativamente similares, se optó por no estandarizarlas. La Figura 1 muestra la distribución de los datos del dataset.

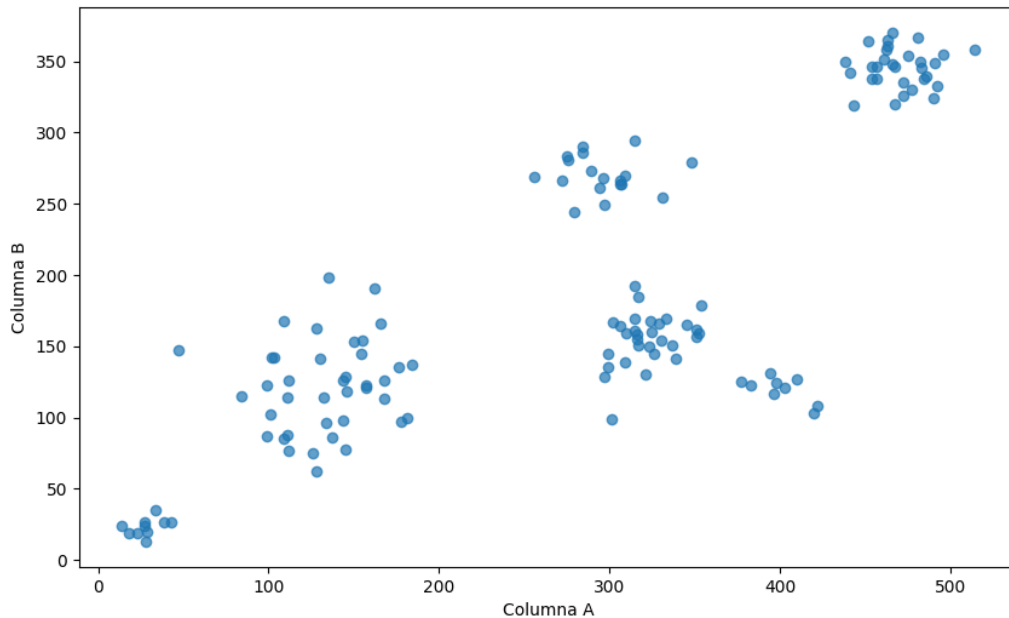


Figura 1: Diagrama de dispersion `data_clusters.mat`.

La Figura 1 permite observar que se pueden diferenciar claramente seis *clusters*; sin embargo, para saber exactamente qué datos están asociados a ellos, se debe implementar el modelo K-means. El proceso propuesto es el siguiente:

1. **Rango de k (número de *clusters*).** Entre 2 y 10.
2. **Inicialización de los k centroides.** Mediante el método *k-means++*, con 50 reinicios. Se mantendrá la misma semilla (*Random State = 42*) para asegurar reproducibilidad. Cada reinicio se entiende como una corrida independiente.
3. **Cambio de centroides.** Mediante el método de Lloyd, con un máximo de 300 iteraciones.
4. **Límite de iteración.** Cuando la inercia (función objetivo) no presente una variación superior a 0.0001 con respecto a su anterior valor en el cambio de centroides, se detendrá la iteración.
5. **Captura de la mejor inercia.** Se registrará el valor más bajo de la inercia (función objetivo) para cada valor de k .
6. **Gráfico del codo.** Con los mejores valores de inercia para cada k , se generará el *gráfico del codo* para elegir el tamaño óptimo de k , que corresponde al punto en el que la curva comienza a aplanarse, es decir, donde aumentar el número de clusters ya no mejora sustancialmente la inercia.

La inercia (función objetivo) se define de la siguiente forma:

$$J(\{C_i\}, \{\mu_i\}) = \sum_{i=1}^k \sum_{j=1}^{n_i} \|x_j^{(i)} - \mu_i\|^2, \quad \mu_i = \frac{1}{n_i} \sum_{j=1}^{n_i} x_j^{(i)}. \quad (1)$$

Donde:

- k : número de *clusters*.
- C_i : conjunto de puntos asignados al *cluster* i .
- $n_i = |C_i|$: cantidad de puntos en el *cluster* i .
- $x_j^{(i)}$: j -ésimo punto asignado al *cluster* i .
- μ_i : centroide del *cluster* i , definido como el promedio de sus puntos.
- $\|x_j^{(i)} - \mu_i\|^2$: distancia euclídea al cuadrado entre el punto $x_j^{(i)}$ y el centroide μ_i .
- J : función de inercia o función objetivo, que el algoritmo busca minimizar.

Tambien es importante mostrar como funciona el algoritmo *K-means++* para elegir los centroides iniciales para cada k . La inicialización de centroides se basa en la siguiente probabilidad:

$$P(x) = \frac{D(x)^2}{\sum_{x' \in X} D(x')^2}, \quad (2)$$

Donde $D(x)$ es la distancia entre un punto x y el centroide más cercano seleccionado hasta el momento.

El procedimiento del algoritmo *K-means++* es el siguiente:

- **Paso 1.** Seleccionar un primer centroide de manera aleatoria entre los puntos del conjunto de datos.
- **Paso 2.** Para cada punto x , calcular $D(x)$, la distancia entre x y el centroide más cercano ya elegido.
- **Paso 3.** Seleccionar el siguiente centroide con probabilidad proporcional a $D(x)^2$, según la Ecuación (2).
- **Paso 4.** Repetir los pasos 2 y 3 hasta obtener k centroides iniciales.
- **Paso 5.** Una vez definidos los k centroides iniciales, continuar con el algoritmo clásico de K-means (asignación de puntos y actualización de centroides).

La Figura 2 muestra el desempeño para los diferentes valores de k .

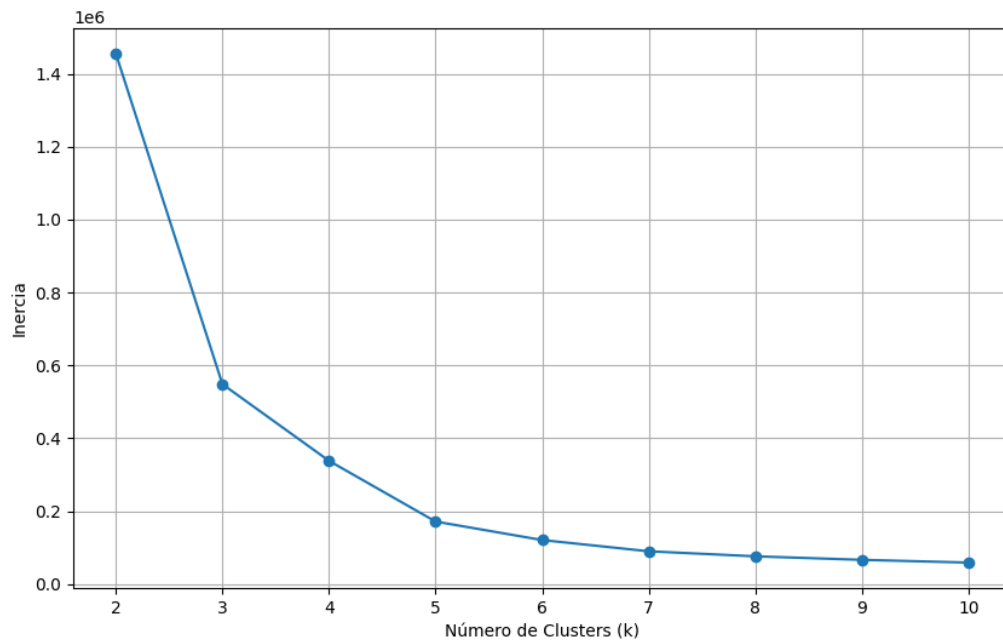


Figura 2: Diagrama del codo para `data_clusters.mat`.

Con el gráfico anterior se concluye que el mejor valor corresponde a $k = 6$. Con esta información, se procede a ejecutar el modelo de *K-means* con dicho valor de k , utilizando los mismos parámetros empleados para su selección. La Figura 3 muestra el resultado de la ejecución de este modelo.

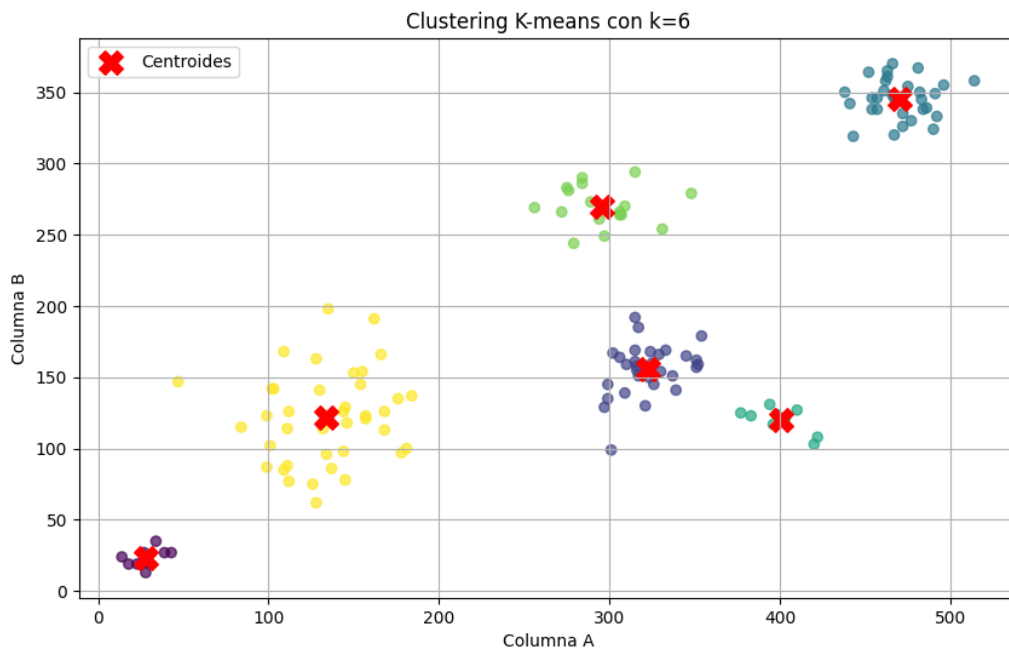


Figura 3: Kmeans `data_clusters.mat`.

La Figura 3 anterior parece haber separado correctamente los diferentes datos en

sus respectivos grupos. No se observa ningún dato que, al menos de manera visual, se encuentre en un *cluster* al que no deba pertenecer.

3.2. SOM con `data_clusters.mat`

De una manera similar a como se procedió en la anterior sección, se plantea la ejecución de un algoritmo SOM estableciendo el siguiente flujo de ejecución con sus respectivos hiperparámetros:

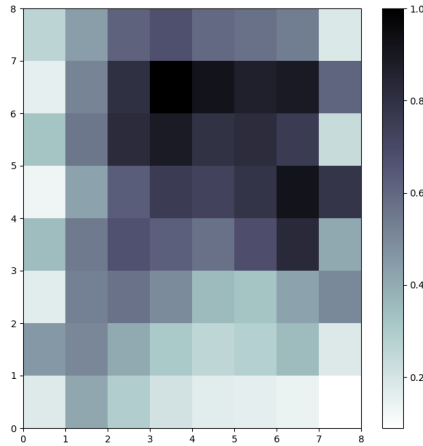
1. **Estandarizar las variables.** las características se normalizan para evitar problemas de escala.
2. **Número de épocas.** se define un total de 100 épocas, divididas en dos fases:
 - *Rough training*: 20 épocas con vecindad grande.
 - *Fine-tuning*: 80 épocas con vecindad pequeña.
3. **Retícula.** Forma cuadrada de 64 neuronas, siguiendo la regla empírica de que el número de neuronas es aproximadamente $5 \cdot \sqrt{n}$, donde n es el número de muestras.
4. **Forma de vecindad.** Se adopta una topología hexagonal.
5. **Tasa de aprendizaje.** α decreciente que inicia en 0.5 y puede llegar hasta un mínimo de 0.01.
6. **Función de vecindad.** Se utiliza una función gaussiana.
7. **Radio de vecindad.** Radio inicial igual a 4 y radio final igual a 1.
8. **Corrección de pesos.** Los pesos se actualizan mediante la regla clásica:

$$w_i(t+1) = w_i(t) + \alpha(t) h_{ci}(t) (x(t) - w_i(t)) \quad (3)$$

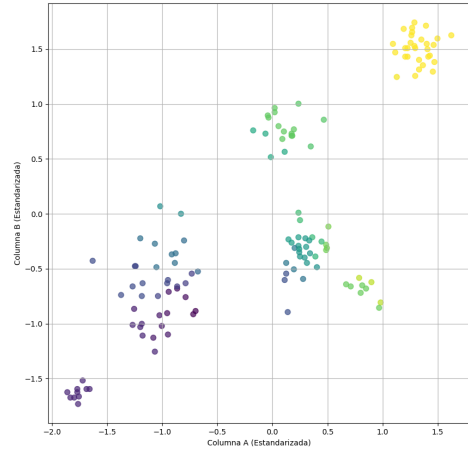
Donde:

- $h_{ci}(t)$: función de vecindad gaussiana.
 - w_i : vector de pesos de la neurona i .
 - $\alpha(t)$: tasa de aprendizaje en el instante t .
 - $x(t)$: vector de entrada en el instante t .
 - t : número de dato que ingresa a la retícula.
9. **Visualización.** Generar gráficos U-Matrix y coloreo de *clusters* para observar el resultado del entrenamiento.
 10. **Evaluación.** Calcular el *Quantization Error (QE)* para evaluar el rendimiento del modelo.

El QE obtenido fue de 0.22, el cual, de acuerdo con la literatura, se puede considerar como un buen ajuste. La Figura 4 muestra, respectivamente, cómo se delimitan las regiones asociadas a los valores de las neuronas y la forma en que se van generando los *clusters*, conforme a la neurona ganadora con la que se relacionan (los datos con el mismo color activan la misma neurona ganadora). Se observa que las neuronas ganadoras parecen clasificar los datos de manera muy similar a como lo hicieron los *clusters* en el algoritmo *k-means*.



(a) Matriz de distancias/similitud.



(b) Activacion de neuronas

Figura 4: SOM resultados data_clusters.mat.

3.3. k-means para Avila's dataset

El archivo que se descarga del enlace [Avila's dataset](#) contiene la siguiente información:

- **avila-description.txt**
- **avila-tr.txt**
- **avila-ts.txt**

Los archivos **avila-tr.txt** y **avila-ts.txt** corresponden a *datasets* sin encabezado (es decir, sin nombres en sus campos). Cada uno tiene un tamaño de 10 430 x 11, donde las columnas se dividen en diez características y una clase en la que se clasifican los registros. Los datos se encuentran estandarizados.

El archivo **avila-description.txt** describe el contenido de los archivos mencionados, así como el contexto de los datos que allí se presentan. Dichos datos registran diversas características de escritura de diferentes autores (cada autor corresponde a una clase).

Las características y la clase son las siguientes:

1. **F2 – Upper margin**: Margen superior.
2. **F1 – Intercolumnar distance**: Distancia entre columnas.
3. **F3 – Lower margin**: Margen inferior.

4. **F4 – Exploitation:** Grado de explotación o uso del espacio.
5. **F5 – Row number:** Número de filas.
6. **F6 – Modular ratio:** Relación modular.
7. **F7 – Interlinear spacing:** Espaciado entre líneas.
8. **F8 – Weight:** Peso.
9. **F9 – Peak number:** Número de picos.
10. **F10 – Modular ratio / Interlinear spacing:** Relación entre el módulo y el espaciado interlineal.
11. **Clase:** Variable categórica que puede tomar los valores A, B, C, D, E, F, G, H, I, W, X, Y.

Antes de implementar el algoritmo **k-means**, se eliminaron los registros correspondientes a las cuatro clases menos frecuentes, tal como se indica en el **Taller_I**. En concreto, se excluyeron los registros pertenecientes a las clases B, C, W y Y. Adicionalmente, se asignaron nombres a las columnas de acuerdo con lo definido en el archivo **avila-description.txt**. Tras la eliminación de registros, los tamaños de los *datasets* **avila-tr.txt** y **avila-ts.txt** quedaron en 10012×11 y 10017×11 , respectivamente.

Para ejecutar el algoritmo **k-means** se utilizó un enfoque similar al aplicado anteriormente, con la diferencia de que se modificaron el número de corridas y el rango de valores de k a probar. En este caso, se establecieron 50 corridas y un rango de k entre 2 y 25. Con la configuración recién descrita, se obtuvo el siguiente gráfico del codo.

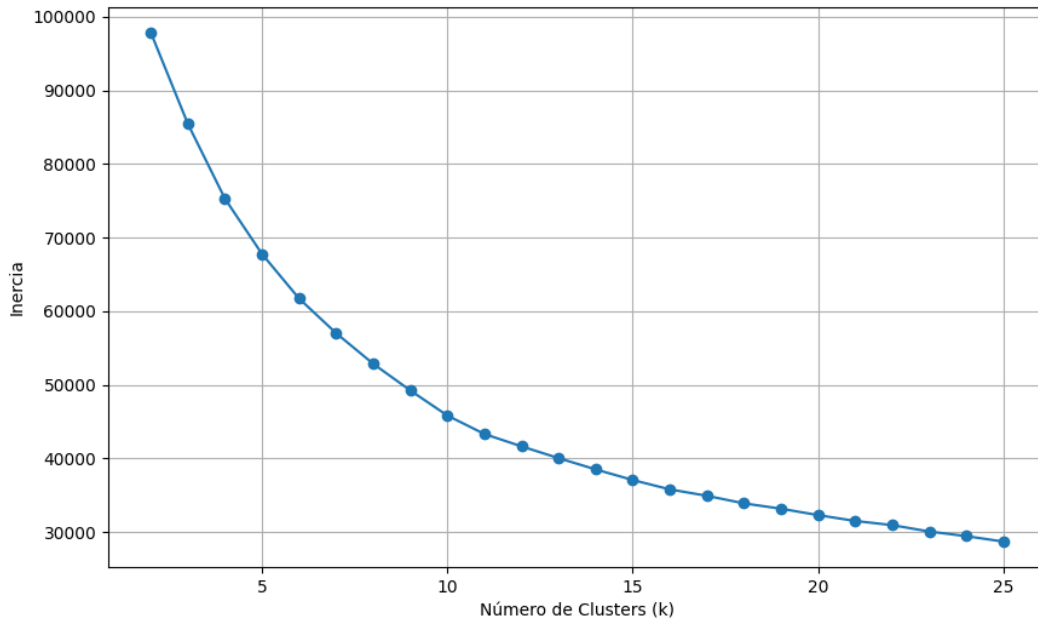


Figura 5: Diagrama del codo para Avila's dataset.

Con el mismo criterio utilizado para seleccionar el valor óptimo de k en el **k-means** anterior, se determinó que $k = 15$ era la mejor elección para este caso. Una vez en este punto ya se puede ejecutar el modelo con la misma configuración con la que se eligió el valor de k .

Con el modelo **k-means** ejecutado, se procede a realizar un proceso de vinculación mediante el cual a cada uno de los *clusters* generados se le asigna una clase. Esta asignación se realiza con base en la clase mayoritaria dentro de cada *cluster*; es decir, se asigna a un *cluster* la clase que más lo representa según la mayor cantidad de registros clasificados en él. Cabe aclarar que en ningún momento se utilizó la información de las clases existentes en los *datasets* para la generación de los *clusters*; la relación entre clases y resultados de **k-means** se estableció mediante el mapeo posicional de cada dato con su respectiva clase en el *dataset* original. Para este proceso se empleó el *dataset* generado a partir de la información contenida en el archivo **avila-tr.txt**.

Con base en lo anterior, es posible generar una matriz de confusion que permita analizar con que precision el algoritmo k-means esta clasificando a las diferentes clases del dataset **avila-tr.txt**. Esto se puede ver en la Figura a continuacion:

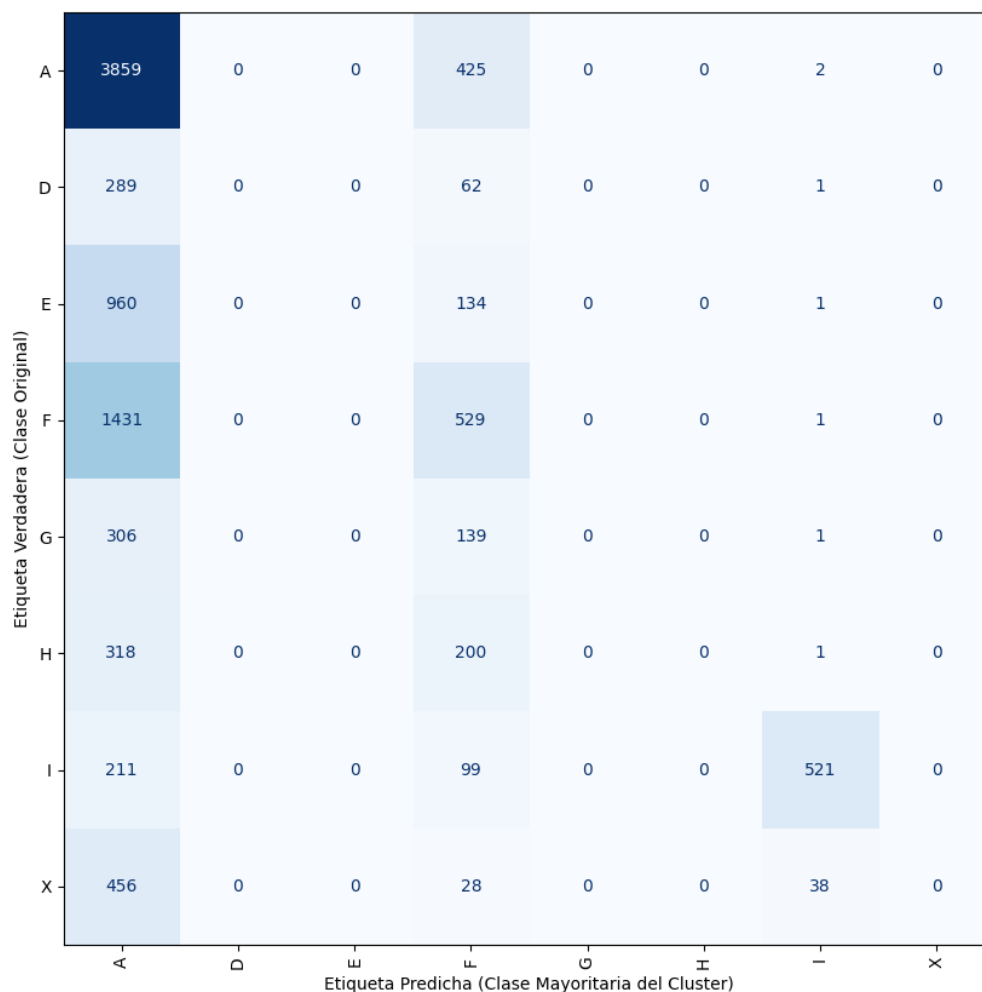


Figura 6: Matriz de confusion para k-means semisupervisado para elAvila's dataset.

Observando la Figura anterior se concluye que el algoritmo **k-means** solo clasifica tres clases: A, F, I, teniendo baja exactitud, pues arroja muchos falsos positivos (especialmente en la clase F), y tiende a confundir mucho la clase A con la clase F, aunque parece gozar de cierta exactitud al clasificar la clase I. Como conclusion se puede afirmar que el algoritmo **k-means** ejecutado tiene un desempeño muy bajo.

Realizando una validacion cruzada del algoritmo en cuestion con el dataset del archivo **avila-ts.txt**, se tiene el siguiente resultado:

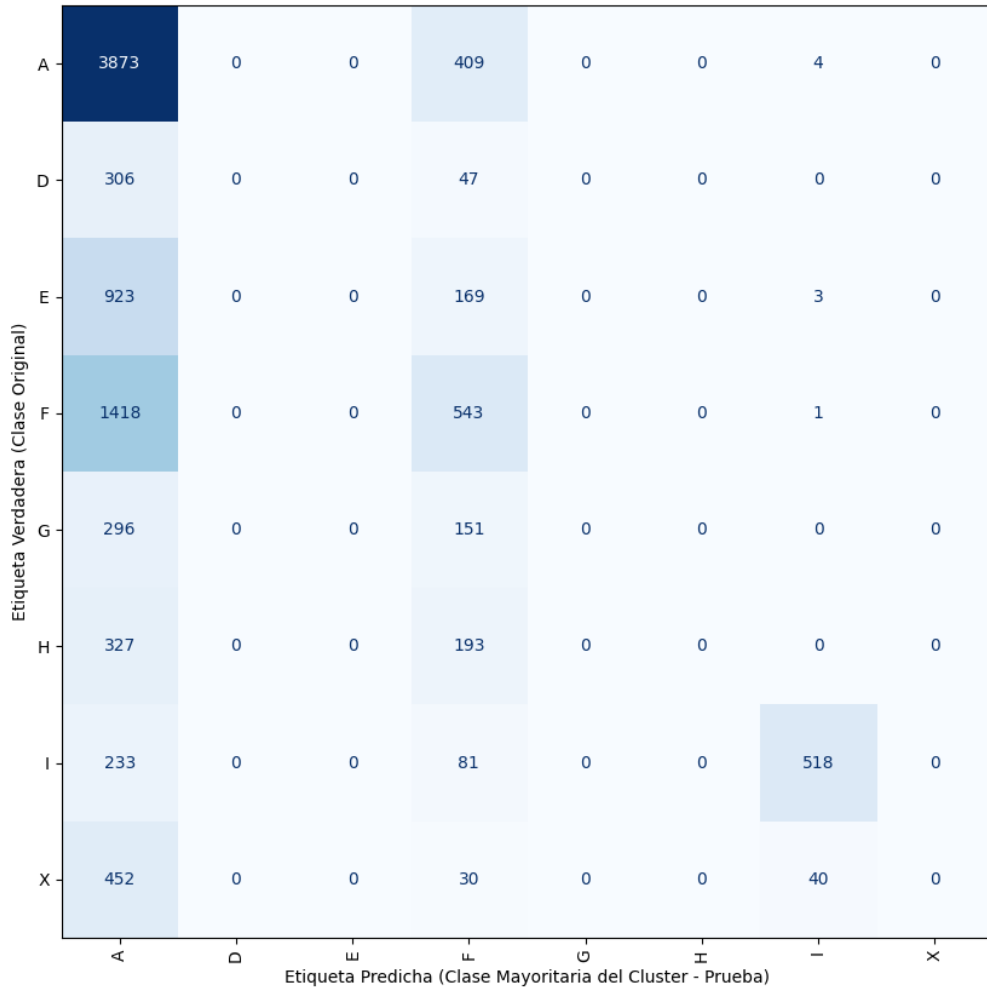


Figura 7: Matriz de confusión para k-means semisupervisado para el **Avila's dataset** dataset de test.

Como lo muestra la Figura 6, el algoritmo **k-means** presenta un desempeño muy similar tanto en el *dataset* de entrenamiento como en el de prueba.

3.4. SOM para Avila's dataset

Para ejecutar el algoritmo **SOM** se utilizó un enfoque similar al aplicado anteriormente, con la diferencia de que se probaron distintos tamaños de retículas. El rango elegido fue entre 16×16 y 36×36 neuronas. Se definió una tasa de aprendizaje que inicia en 0.4 y finaliza en 0.2, un radio de vecindad variable de acuerdo con el tamaño de la retícula evaluada, y se empleó un algoritmo de aprendizaje **batch SOM**, el cual funciona de manera similar al mecanismo tradicional de corrección de pesos del SOM, pero en lugar de ajustar los pesos de las neuronas en cada entrada, realiza la corrección en bloque; es decir, con todos los datos que ingresan en una época. Los resultados del entrenamiento (ver Figura 8)

permiten concluir que la retícula de tamaño 29×29 es la que arroja el mejor desempeño en relación con el valor de QE.

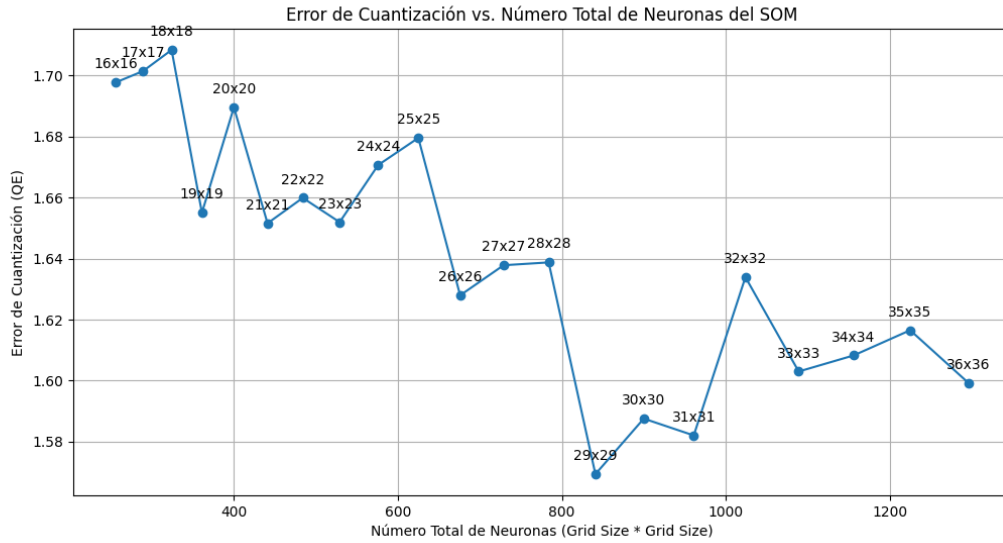


Figura 8: QE versus tamaño de retícula.

Teniendo el tamaño óptimo de la retícula, de acuerdo con el entrenamiento realizado sobre el *dataset* de entrenamiento, se procede a ejecutar el modelo SOM. De manera similar a lo realizado con el algoritmo **k-means**, se genera un mapeo de los datos con su respectiva clase, y se construye una matriz de confusión (ver Figura 8) que permite evaluar el desempeño del modelo y adicionalmente se tienen las siguientes métricas:

Métrica	Valor
Accuracy	0.5574
F1-Score (Ponderado)	0.5007
Precisión (Ponderado)	0.5426
Recall (Ponderado)	0.5574

Tabla 1: Métricas de clasificación SOM dataset de entrenamiento.

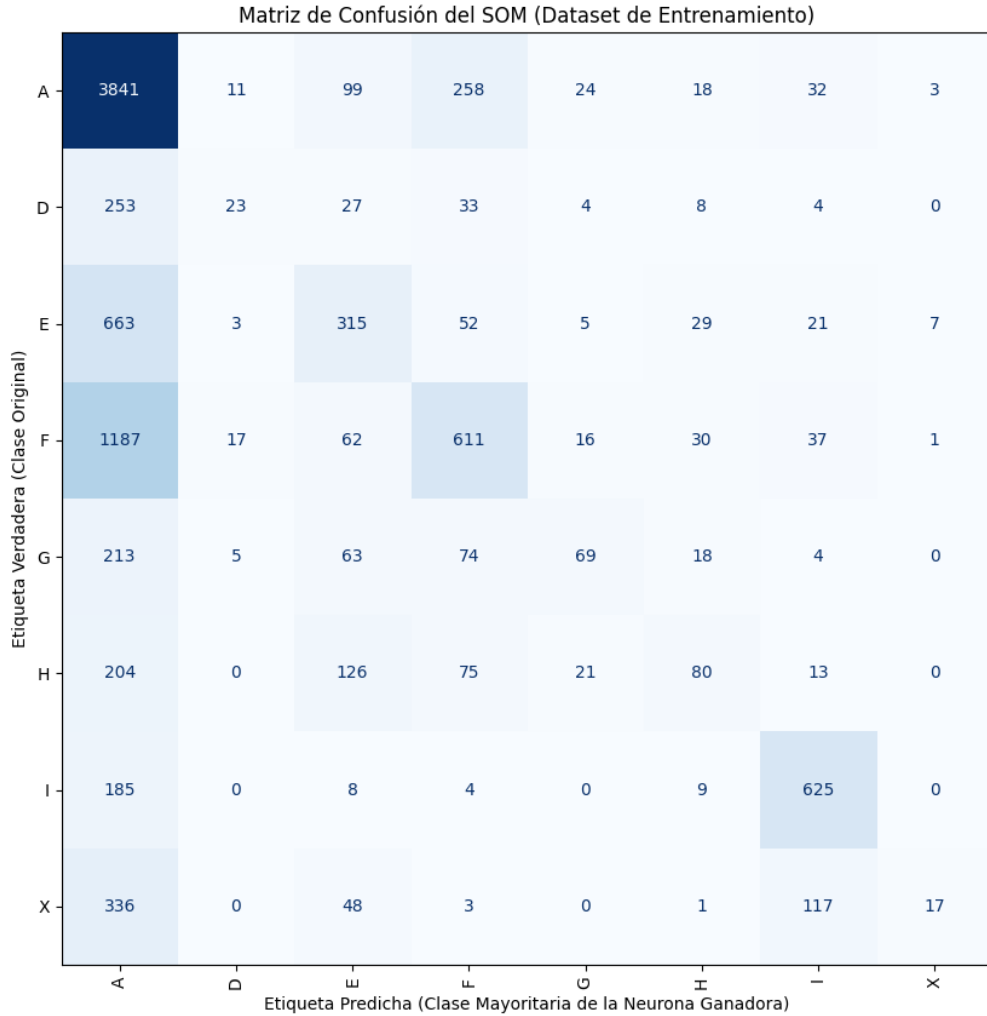


Figura 9: Matriz de confusion para SOM semisupervisado para el dataset de entrenamiento.

Analizando los resultados, se observa que el modelo presenta un desempeño limitado. Sin embargo, en contraste con el algoritmo **k-means**, el modelo **SOM** logra capturar un mayor número de clases, aunque aún con bajo rendimiento; incluso consigue identificar todas las clases del *dataset*. Es posible que, con un *dataset* más balanceado, el modelo **SOM** pueda ofrecer mejores resultados. Asimismo, sería interesante probar un análisis de características previo a su uso en el modelo, ya que algunas variables podrían no estar aportando información relevante, generar ruido o presentar colinealidad con otras.

También se hizo un proceso de validación cruzada en donde se probó el modelo entrenado **SOM**, con los datos del dataset del archivo **avila-ts.txt**. Se obtuvo el siguiente resultado:

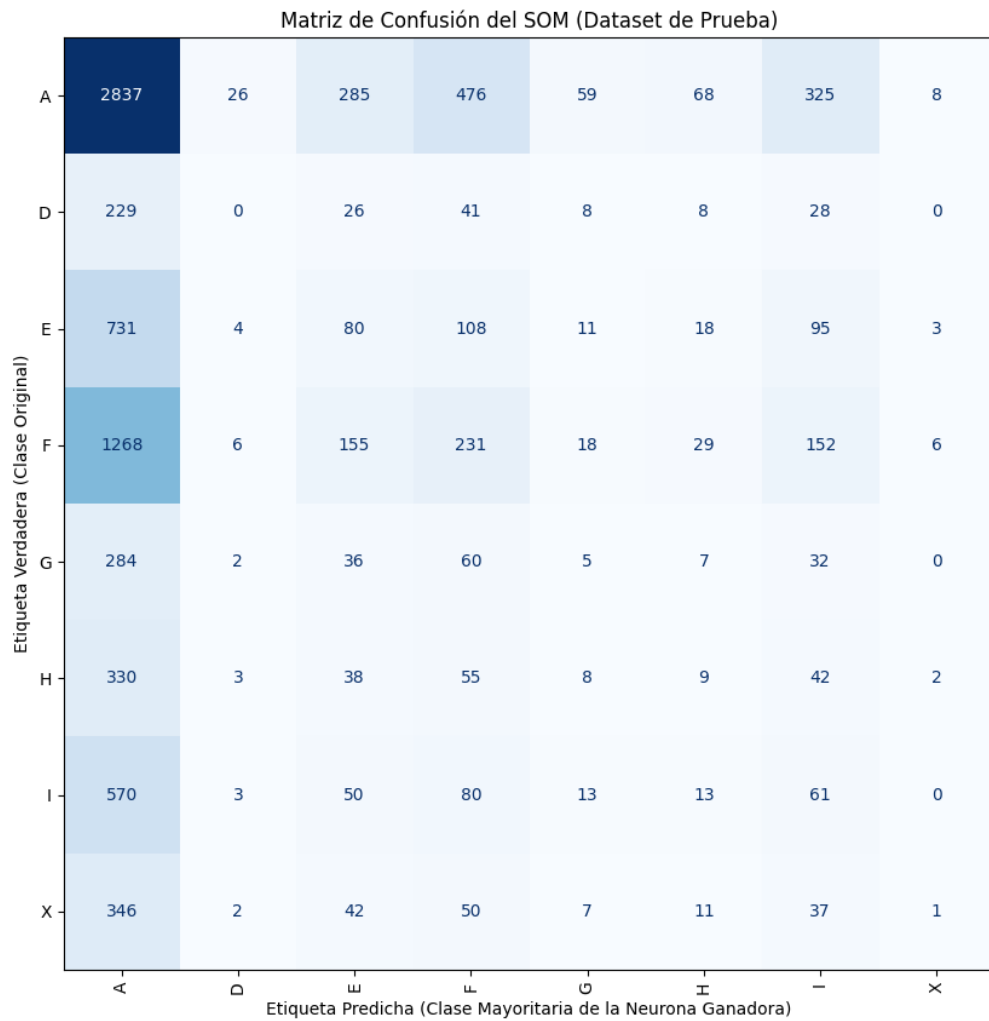


Figura 10: Matriz de confusion para SOM semisupervizado para el dataset de test.

La Figura 10 arroja que el modelo se comporta de manera muy similar a como se comportó en el conjunto de entrenamiento.